

Graph

directed graph: strongly connected.

undirected graph: connected components.

weighted graph: If we do not mention the graph is weighted, the graph is unweighted.

BFS

time complexity $\Theta(|V| + |E|)$

implement by queue.

DFS

time complexity $\Theta(|V| + |E|)$

implement by stack.

Disjoint Set Union

Union-by-height: The worst case tree of height h must result from taking union of two worst case trees of height $h-1$.

Worst-Case Scenario

Thus, suppose we have a worst-case tree of height h

- The number of nodes is $\sum_{k=0}^h \binom{h}{k} = 2^h = n$
- The sum of node depth is $\sum_{k=0}^h k \binom{h}{k} = h2^{h-1}$
- Therefore, the average depth is $\frac{h2^{h-1}}{2^h} = \frac{h}{2} = \frac{\lg(n)}{2}$
- The height and average depth of the worst case are $O(\lg(n))$

path compression:

Time complexity

With both optimization methods, could it be any better than $O(\ln(n))$?

– is there something better?

The amortized time complexity is $O(\alpha(n))$ where $\alpha(n)$ is the inverse of the function $A(i, i)$, that is

$$\alpha(n) = \min \{i \mid A(i, i) \geq n\}$$

where $A(i, j)$ is the Ackermann function:

$$A(i, j) = \begin{cases} j+1 & \text{if } i = 0 \\ A(i-1, 1) & \text{if } i > 0 \text{ and } j = 0 \\ A(i-1, A(i, j-1)) & \text{if } i > 0 \text{ and } j > 0 \end{cases}$$

The first values are:

$$A(0, 0) = 1, \quad A(1, 1) = 3, \quad A(2, 2) = 7, \quad A(3, 3) = 61$$

$\alpha(n)$ can not be ignored.

MST

(中文是解释, 不是keypoints)

Prim's

Cut property: Divide $V = S \cup (V \setminus S)$, if one edge $e = (u, v)$ satisfies $u \in S$ and $v \in V \setminus S$. Then we call e as a cut of S and $V \setminus S$.

Denote MST $T = (V, E')$ and the set of cut edges between S and $V \setminus S$ as $cut(S)$. Then $\forall S \subset V, \arg \min_{e \in cut(S)} w(e) \in T$. i.e. For all vertex subsets S , the edge with minimal weight in $cut(S)$ is in MST.

We can use the cut property to get a (greedy) algorithm: Prim's

Choosing one vertex -> Finding the minimal cut edge -> Adding the new vertex into the set -> Repeat until no new vertex.

How cut property works: "Cut" -> Tree, cut property -> Minimal

(这里的意思是, “切分”本身保证了出来的一定是树, 切分性质保证了它最小。)

Time complexity: $O(|V|^2 + |E|) = O(|V|^2)(|V|)$ for traversing **each vertex** to get minimal, $|V|$ is the repeating time)

Binary heap: $O((|V| + |E|) \log |V|) = O(|E| \log |V|)(|E|)$ for traversing **each edge** for heap, $\log |V|$ is time taken to reduce key.)

(这段PPT上说的是log时间将heap元素减小, 表述上有点奇怪, 但是结论不变, 常规的ElogV是每条边都要会被加入那个堆里, 而堆的大小是 $|V|$, 次数是 $|E|$)

Fib heap: $O(|E| + |V| \log |V|)$ (always better)

Notice: Binary heap does not always optimize the time, especially for dense graphs $|E| = \Theta(|V|^2)$
 . Fib heap optimizes insertion from $\log \rightarrow 1$

Kruskal's

Another greedy, but from edges.

Sort the edges by ascending weights -> Consider each edge in order, if it does not conflict with the recent subgraph, add this edge into MST -> Repeat until $|V| - 1$ times of add.

Brief Proof (不用写上去, 可讲) : If there exists a better one, consider the first different edge that Kruskal's will not choose, and add to the tree Kruskal's spanning graph. Then it will create a loop. If their weights are not the same, then the maximal edge on the loop must be the different edge.

Time complexity: $O(|E| \log |E| + |E||V|) = O(|E||V|)$. ($|E| \log |E|$ for sort and $|E||V|$ for $|E|$ times choosing edges, one cost $|V|$ to find loop.)

After disjoint set: $O(|E| \log |E| + |E|\alpha|V|) = O(|E| \log |E|)$. ($|E| \log |E|$ for sort and $|E|\alpha(|V|)$ for $|E|$ times cost $\alpha(|V|)$ to find loop.)

If there exists a $\Theta(|E|)$ sorting, $\Theta(E\alpha|V|)$ (存疑, PPT上说 $|E|$)

Algorithm	Prim's	Kruskal's
Trivial time	$O(V ^2)$	$O(E V)$
Optimized data structure	Binary/Fibonacci Heap	Disjoint sets
Optimized time	$O(E \log V)/O(E + V \log V)$	$O(E \log E)$